OCTO-BEE User Guide

High Performance Simultaneous Data Acquisition

OCTO-BEE-CELF/OCTO-BEE-CONC

Hampton 3D Field Measurement System
Installation and User Guide

Revision History

Revision	Date	Author(s)	Description
1	17/09/2025	EM	Created
2	18/09/2025	EM	Added Detailed Phoebus Installation
3	22/10/2025	EM	Added Plotting Data, I2C ADC Communication and proposed Sample Data format (See Section 4, 6 and Section 7)
4	27/10/2025	EM	Added Encoder Information. See Section 2.3
5	28/01/2026	EM	Finalised to Final Data Packet and added additional testing done
6	06/02/2026	EM/SR/CH	Added contents to Appendix on how to upgrade to ”Gold System”

Table Of Contents

Glossary

- CONC : Concentrator
- FMC : VITA57.1 FPGA Mezzanine Card
- ELF : Electrically Extended FMC, implies ULPC or DULPC (only compatible with D-TACQ carriers)
- LPC : FMC Low Pin Count standard as per VITA57.1
- ULPC : Subset by D-TACQ, Ultra Low Pin Count
- DULPC : Subset by D-TACQ, Differential Ultra Low Pin Count (ULPC with extra differential signalling)
- Xilinx ZYNQ System on Chip (SoC)
- FPGA : Field Programmable Gate Array

⚠ WARNING

- **The system should be disconnected from the mains supply and ESD precautions taken before attempting to touch/connect the OCTO-BEE system.**
- **The system must be powered down before unplugging or plugging in any Evaluation Kits.**
- **The system must be powered down before disconnecting or connecting any cables from the Concentrator board.**
- **The system must be powered down before unplugging or plugging in any encoders to the rear terminal connectors.**

1 System Overview

The **OCTO-BEE system** consists of the following components:

- ACQ1001S
- ACQ423-FFC
- OCTO-BEE-CELF
- OCTO-BEE-CONC
- SENIS Evaluation Kit(s)

Figures 1 and 2 show the complete OCTO-BEE system, including the connections between the CELF, the CONC, and the SENIS Evaluation Kit.

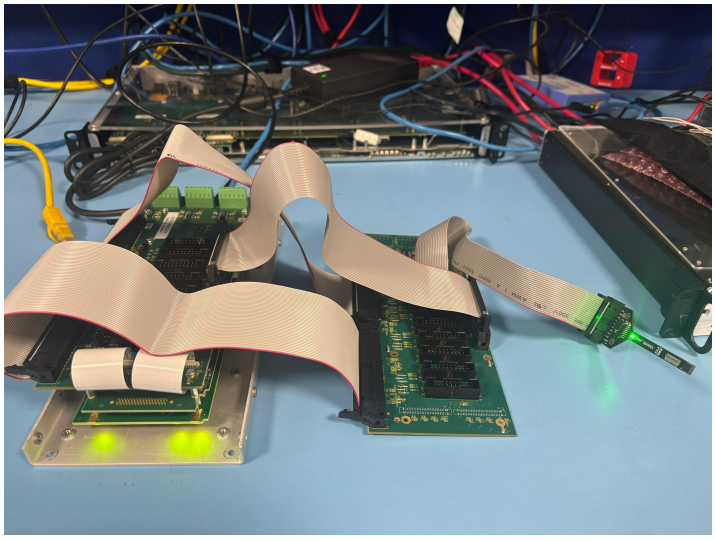


Figure 1: OCTO-BEE System – Front View

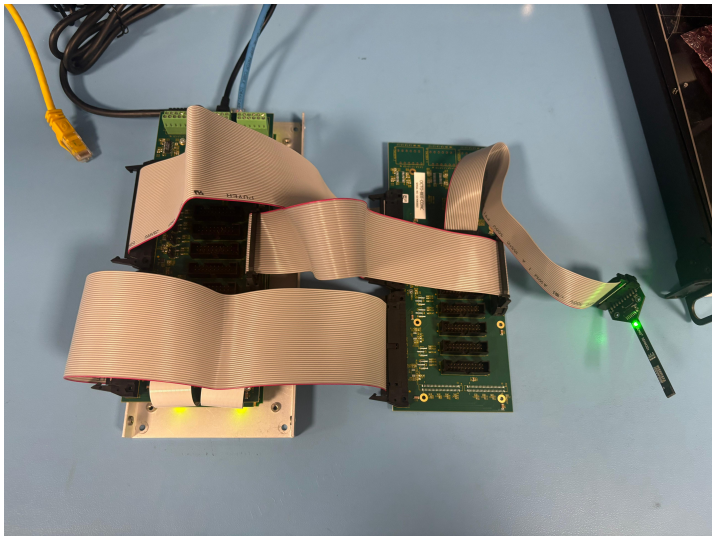


Figure 2: OCTO-BEE System – Aerial View

Note: D-TACQ currently has access to only one SENIS Evaluation Kit, connected to Port 1 as shown in the figures above. In the final configuration, up to **eight evaluation kits** are expected to be connected.

2 Test Setup

2.1 Required Equipment and Connections

- 1x – 34-Way Ribbon Cable
  - Provides digital and power connection from CONC to CELF.
  - D-TACQ has supplied a short version of this cable for test purposes.
  - For the final Hampton design, it is assumed that Hampton will supply their own cable at the required length.
- 2x – 50-Way Ribbon Cable
  - Carries analog channels from CONC to CELF.
  - D-TACQ has supplied short versions of these cables for test purposes.
  - For the final Hampton design, it is assumed that Hampton will supply their own cables at the required length.
- At least 1x – 20-Way Ribbon Cable
  - A total of 8 cables are required for the complete “OCTO” system.
  - Connects SENIS Evaluation Kit(s) to CONC or CELF.
  - D-TACQ has not supplied these cables.

2.2 Setup Instructions

2.2.1 Local Connection of Evaluation Kits

For a “quick connection,” the SENIS chip can be connected directly to the CELF module, as shown in Figure 3.

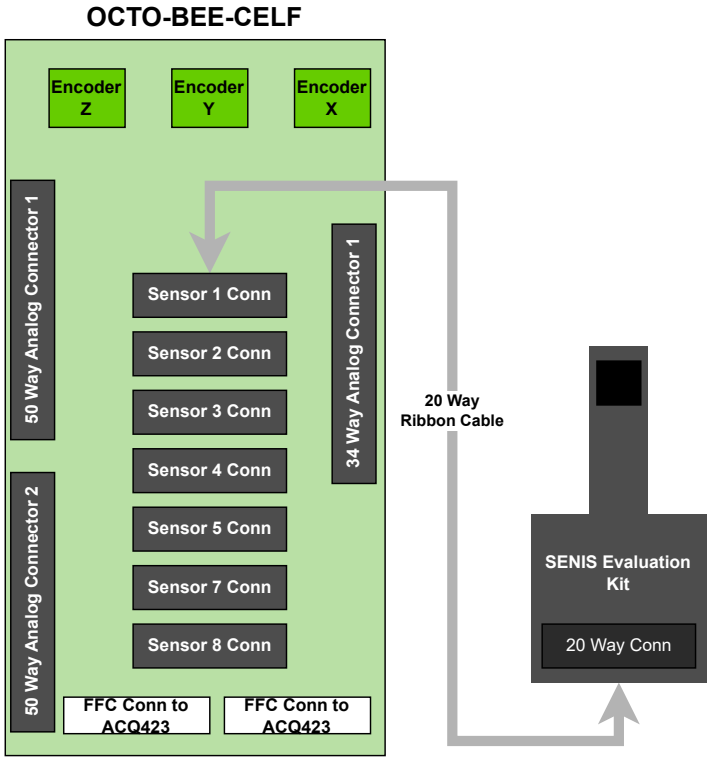


Figure 3: OCTO-BEE Quick Path Diagram

Instructions:

1. Ensure the system is **powered off**.
2. Connect a 20-way ribbon cable from the header of the SENIS Evaluation Kit to the desired port on the CELF.
3. Once all required Evaluation Kits are connected, power on the device.
4. Verify that the green LED on each Evaluation Kit is illuminated.

2.2.2 Full System Assembly

For full system operation, the Concentrator board (OCTO-BEE-CONC) is connected to the CELF, as illustrated in Figure 4.

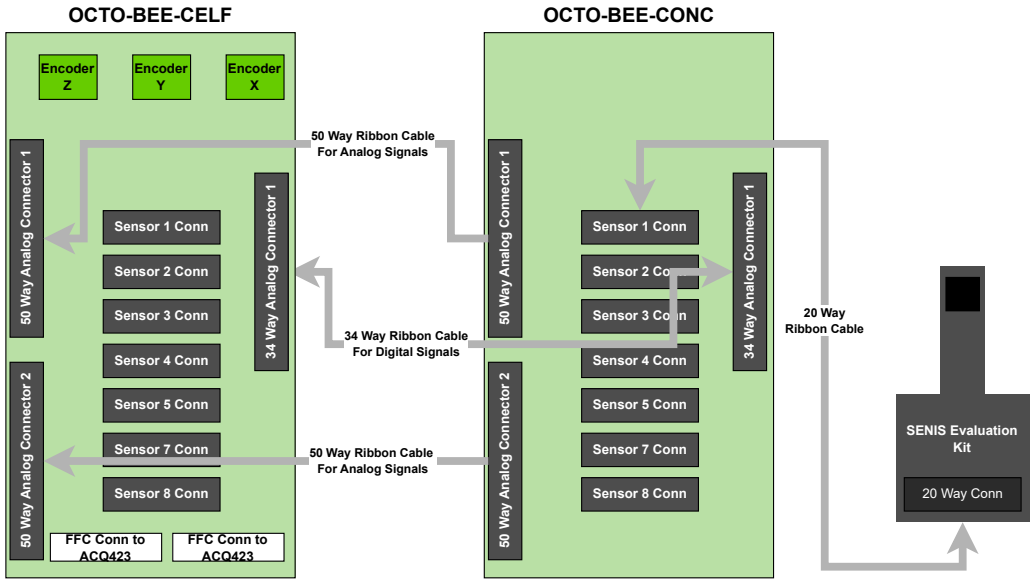


Figure 4: OCTO-BEE Full Path Diagram

Instructions:

1. Ensure the system is **powered off**.
2. Connect a 20-way ribbon cable from the header of each SENIS Evaluation Kit to the desired port on the CONC.
3. Connect the corresponding ribbon cables from the CONC to the CELF, as shown in Figure 4.
4. Once all required Evaluation Kits are connected and the CONC is linked to the CELF, power on the device.
5. Verify that the green LED on each Evaluation Kit is illuminated.

Whether you configure the system using the CONC or directly to the CELF you should see no different results and testing is identical.

2.3 Encoder Wiring

The encoder designed to is **GHB38-08G3600BML5**.

We have designed to the Line Driver output variant.

Please ensure system is powered off when wiring the encoder. To connect the wires:

Loosen the screw on the connector for the pin you wish to wire, just enough so that the top of the screw is free but still engaged. Insert the wire into the corresponding hole, then tighten the screw until the wire is secure and cannot be pulled out. Repeat this process for all eight required wires.

Ensure the exposed metal of the wires are not touching each other.

Table 1 summarizes the wiring scheme for the GHB38 encoder. Refer to Figure 5 for a visual representation of the wiring layout. The PCB silkscreen indicates the corresponding signal for each pin.

Wire Colour	**Signal**
Red	Power Supply VCC (DC4.5V 30V)
Black	Power Supply GND (0V)
Green	Output A Phase
Brown	Output -A Phase
White	Output B Phase
Grey	Output -B Phase
Yellow	Output Z Phase
Orange	Output -Z Phase

Table 1: Optical Incremental Encoder GHB38 Wiring Scheme

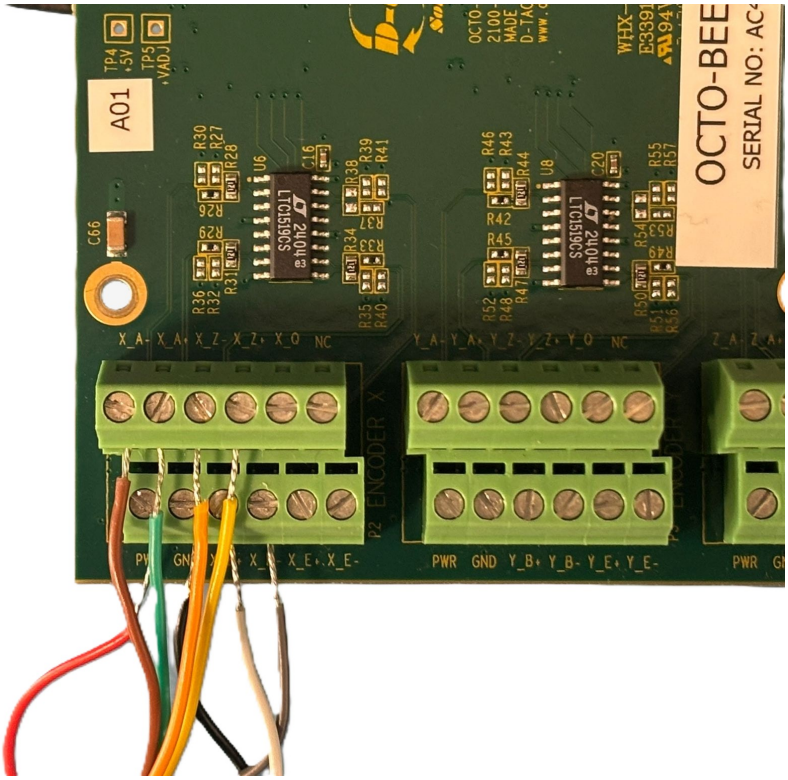


Figure 5: Encoder Connector Wired Up

3 Acquisition Quick Start

3.1 Setup - Host Software

3.1.1 CS-Studio Phoebus GUI

New, developing replacement for traditional CS-Studio.
Please follow instructions at https://github.com/D-TACQ/ACQ400CSSP

If required, for a full guide on each step please see Appendix Section A

3.1.2 ACQ400 HAPI Installation

Python code and full install instructions can be retrieved from : https://github.com/D-TACQ/acq400_hapi

To install :

```
mkdir PROJECTS; cd PROJECTS
git clone https://github.com/D-TACQ/acq400_hapi.git
cd acq400_hapi
```

on Linux , run `source ./setpath`
on Windows, run `SETPYTHONPATH.BAT`

Make sure and restart your shell on Windows after running the BAT file.

For an explanation of how to use HAPI to gather data from the box please see Section 4.

3.2 Phoebus - Recommended Windows

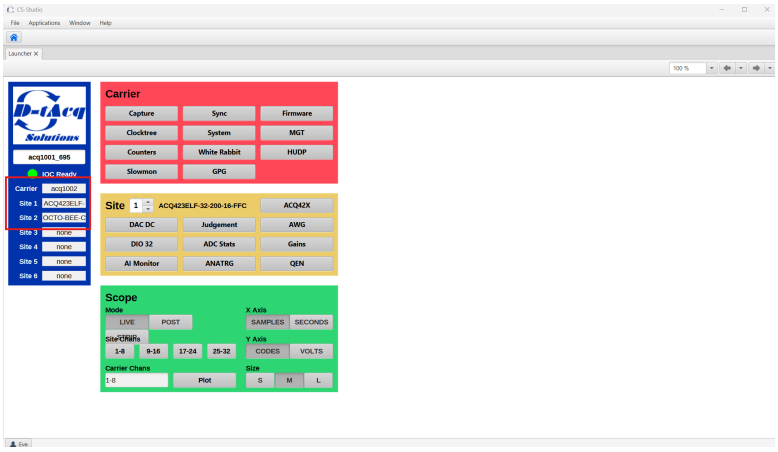


Figure 6: CSS Phoebus ACQ400 Launcher

3.3 Clocktree and Counters

System will ship from factory running on internal clock and trigger.

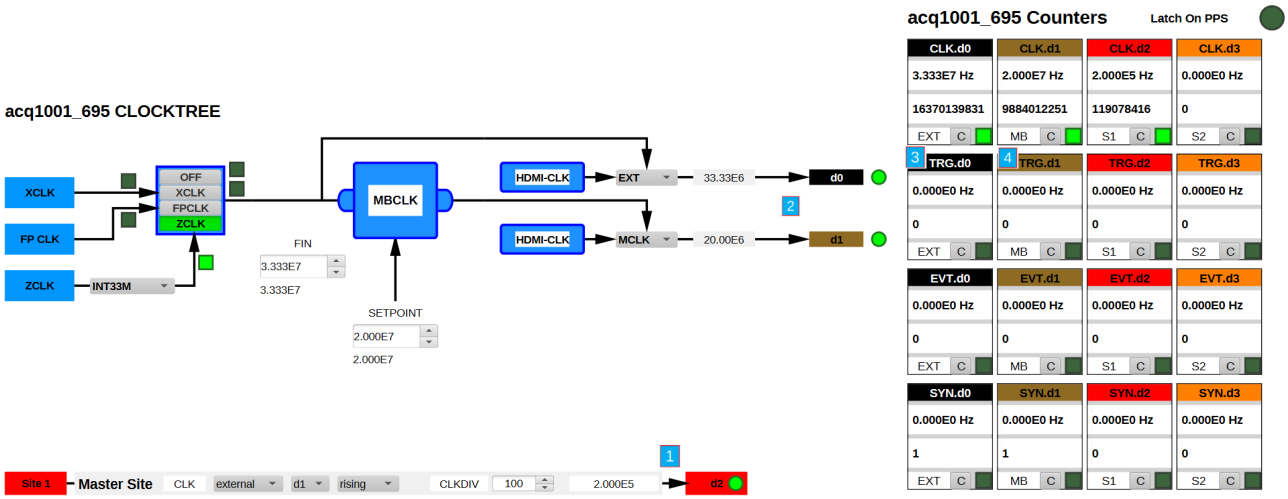


Figure 7: Clocktree and Counters OPIs

- 1 Site 1 ADC Clock 200 kHz
- 2 Clock bus d0 and d1 are motherboard level clocks
- 3 Count TRG.d0 (default Front Panel) triggers
- 4 Count TRG.d1 (default Soft) triggers

3.4 Streaming Capture - IDLE

Figure 8: CSS Phoebus Capture OPI - Idle

- 1 Soft-trigger selected by default
- 2 ADC sites in Aggregator set
- 3 Scratchpad (SPAD) LWords (digital signature containing sample counter, timing info)
- 4 Sample Size in Bytes (SSB); combination of ADC channels and optional SPAD
- 5 Stream START/STOP button

3.5 Streaming Capture - ACTIVE

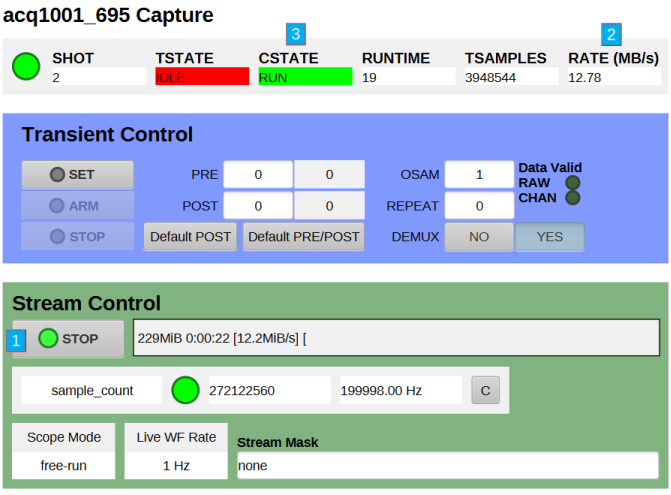


Figure 9: CSS Phoebus Capture OPI - Active

- 1 Stream START/STOP button
- 2 Data Rate, 12.8 MB/s
- 3 Continuous State

This provides a quick stream to have a quick look at the data live. Before starting a stream where “offloading” data is required as in Section 4 please ensure this is switched **OFF**.

3.6 Streaming Live Plot - Setup

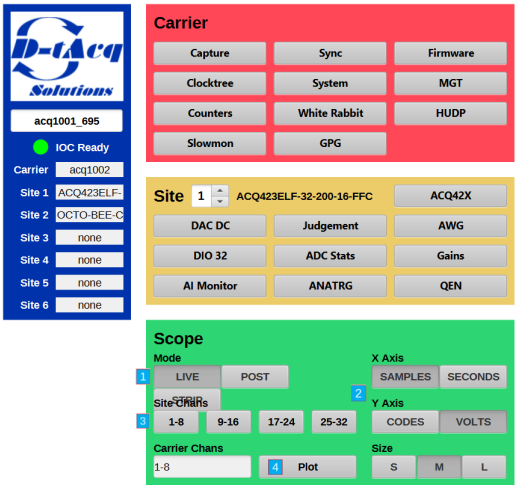


Figure 10: CSS Phoebus Live Plot View

- 1 Live or Post Shot button
- 2 Axis Control
- 3 What Channels to view (in sets of 8)
- 4 View Plot Button

3.7 Streaming Live Plot - View

Live monitor of ADC sampling.

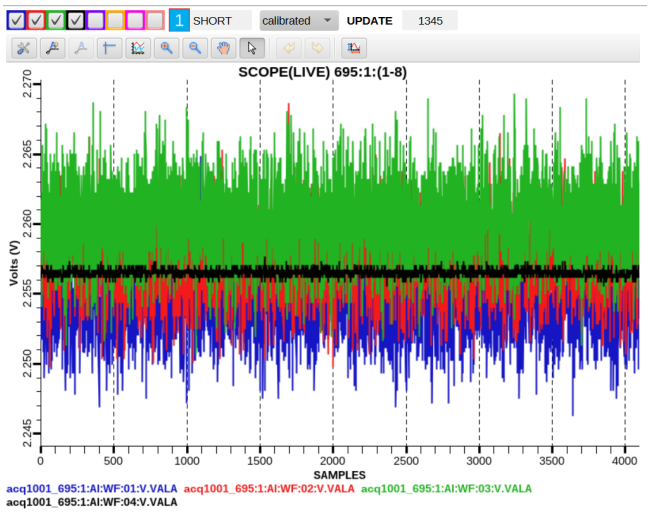


Figure 11: CSS Phoebus Live Plot

- 1 Hide/View Channels

Each Evaluation kit uses four consecutive channels: the first is Z-axis, the second is Y-axis, the third is X-axis, and the fourth is VCM, so SENIS kit 1 maps to channels 1–4, kit 2 to channels 5–8, kit 3 to channels 9–12, and so on.

Now you can put near a magnet and view relevant axis changing as shown in the verification report.

4 Gathering & Plotting Data

4.1 Gathering Data

You can gather data using HAPI on a host machine, **after ensuring you've installed ACQ400 HAPI as detailed in 3.1.2.** To gather data so it can then be examined / plotted open a command line in the directory **acq400_hapi/user_apps/acq400** or navigate to this directory and run the command:

```
python acq400_stream.py --verbose=1 --filesize=1011712 --runtime=10 acq1001_694
```

This simple mode of acquisition sets up and operates a stream to host from socket 4210 on the OCTOBEE system. The files are saved by default to the paths:

```
$UUT/000001/0000
...
$UUT/000001/0010
```

In our case:

```
acq1001_694/000001/0000
...
acq1001_694/000001/0010
```

To see the in-built help for a HAPI script (this works for most scripts) you can run the command:

```
python acq400_stream.py -h
```

4.2 Plotting Data

An example script on plotting this data is host_demux.py.

Navigate or open a command line in **acq400_hapi/user_apps/analysis** and run the following command:

```
python host_demux.py acq1001_695 --src=../acq400/acq1001_695/000001/ --pcfg=../../PCFG/octo_bee.pcfg
```

This script accepts an argument for a plot configuration file (.pcfg), which allows you to tweak and label channels. Our configuration file is saved two directories above the analysis directory, inside a folder named PCFG. Therefore, the argument for the script is - -pcfg= followed by the file path. You can move the configuration file and adjust the path accordingly to match its location.

This plot shows analog channels 1–4 in volts, temperature reading in codes and the sample counter.

Channels 35 and 36 together represent the sample counter, which increments with each sample. Channel 35 displays the lower portion of the counter, while channel 36 displays the upper portion.

Because the counter is being handled in two short parts (16-bit data), the lower portion (channel 35) ramps steadily from its minimum to its maximum value, then rolls over to zero—resulting in a sawtooth-like waveform. Each time that rollover occurs, the upper portion (channel 36) increments by one step, so it appears as a staircase or stepping waveform.

If you were instead to plot the counter as a single 32-bit value, you would see just a single smooth ramp. See Section 7 for more information on the data format.

See output plot in Figure 12:

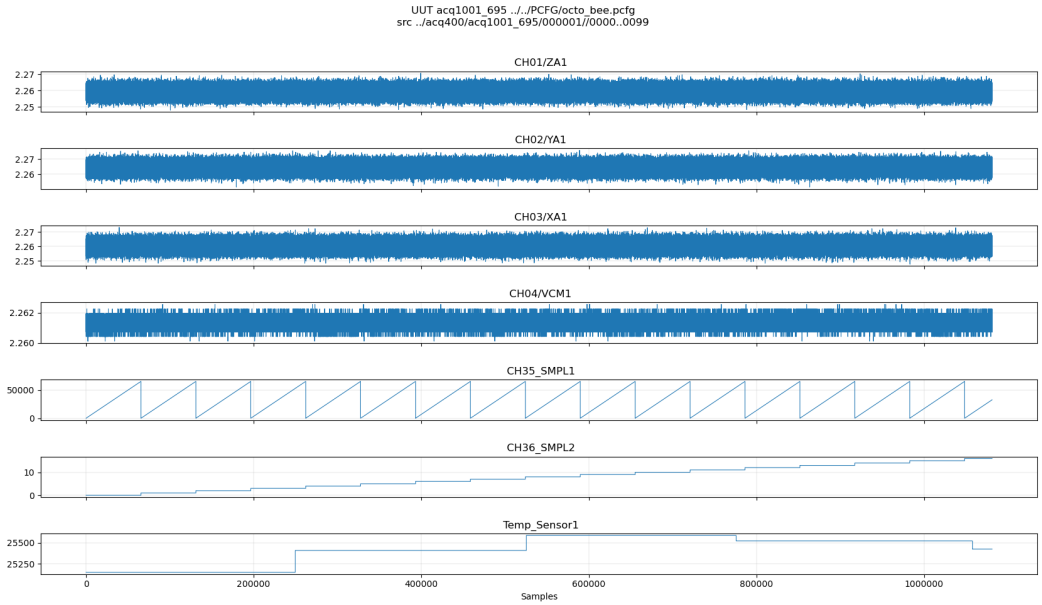


Figure 12: Plot of Channels 1-4 and Sample Rate

4.3 Example Plotting Data 8 Sensors

An experiment was conducted at D-TACQ using eight sensors to demonstrate a representative application, as shown in Figure 13.

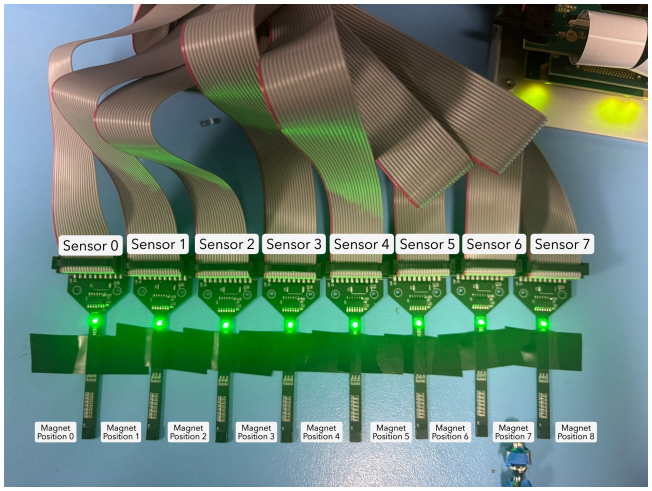


Figure 13: 8-Sensor Experiment Setup

The sensors were aligned side by side with uniform spacing. A magnet was sequentially placed at positions 0 through 8, and a short capture of all sensor data was recorded at each position.

As the magnet was located between adjacent sensors, each device measured the magnetic field along the y-axis. The resulting data were mapped to the heatmap in Figure 14, showing the y-axis response of each sensor for every magnet position.

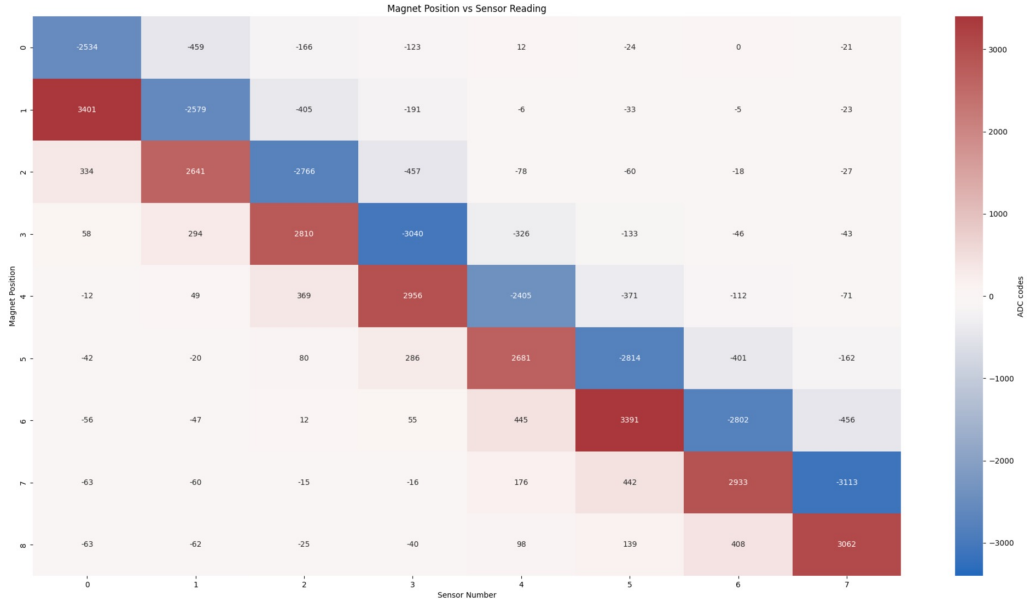


Figure 14: Plot of 8-Sensor Experiment Results

The results are strongly dependent on the magnet strength. All sensors were configured with maximum gain, so a stronger magnet would be expected to increase the response amplitude, particularly for sensors further from the magnet. The ADC data are plotted in raw codes, with the VCM for each sensor subtracted during post-processing.

4.4 Encoder Test

We have an encoder on the X and Y axis. A short stream was done while turning the X and Y encoder clockwise.

For gathering the data we use ”acq400_stream.py” again as shown below:

```
python /user_apps/acq400/acq400_stream.py --totaldata 104000000 --filesize 1011712 --verbose 1 --root Encoder_ acq1001_695
```

This records a short burst of data and saves it in a directory called ”Encoder”.

To plot this data we use ”host_demux.py” again, to plot all of the encoder channels in 32 bit. Note there is no encoder on the Z-axis hence no movement.

```
python user_apps/analysis/host_demux.py acq1001_695 --data_type=32 --nchan=26 --pchan=17,18,19 --src=Encoder/000001
```

See output below

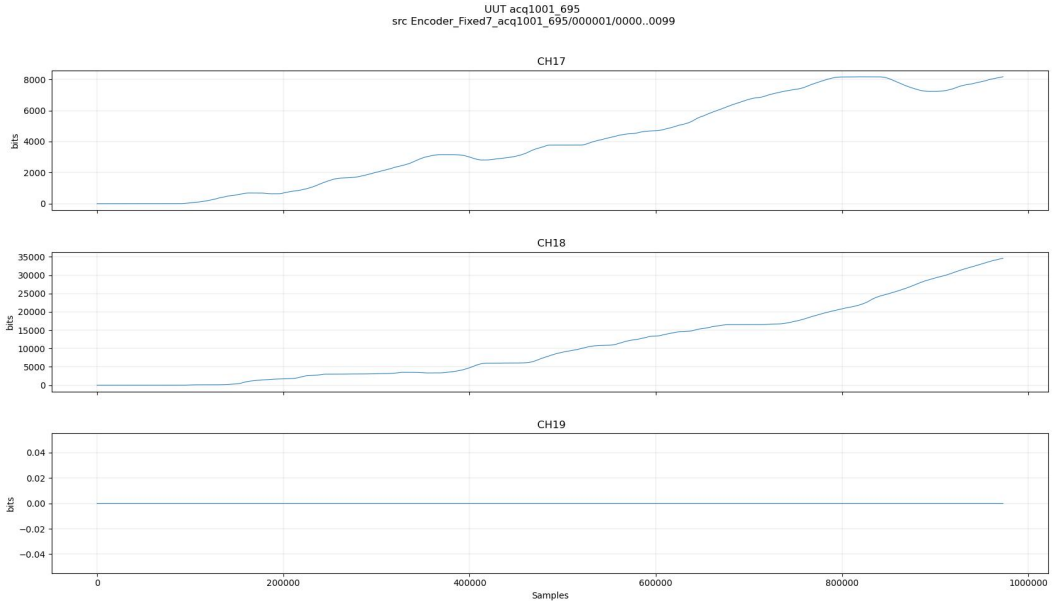


Figure 15: Plot of Encoder Axis

5 SPI Setup and Commands

5.1 Detailed SPI Setup Instructions

D-TACQ has ported the SPI Interface *python* functions provided by SENIS to the ACQ400 software platform. Initially the SPI Interface was tested with just one Development Board which worked as expected after some modifications of the SENIS software configuration to the D-TACQ microprocessor.

The full suite of commands described in the SENM3Dx-EKv2.4 – 3DHALL EVALUATION KIT Preliminary Manual (Rev.1.1 October 2024) is available to setup and control the Sensor when plugged into the OCTO-BEE board. This is shown below:

The key setup commands are:

```
>>> import ThreeDHALLInterface as tdhi
>>> dx = tdhi.ThreeDHALLInterface()
>>> dx.openSPI(1,0x20)
>>> dx.configureSPI('976kHz')
```

The `openSPI` sets the Sensor Position on the OCTO-BEE to which the SPI commands will be targeted as follows:

```
Number       Sensor Position
0x20         1
0x21         2
0x22         3
0x23         4
0x24         5
0x20         6
0x26         7
0x27         8
```

In the above example it is connecting the SPI to Sensor Position 1.

Now you can run the commands provided by SENIS for any additional calibration needed.

5.2 SPI 8 Sensors ”Quick Test”

For a quick check that all 8 Sensor’s SPI is verified you can run this command on the box:

```
acq1001_695> /usr/local/CARE/senis-test.py
```

This simply initialises and verifies SPI for all 8 sensors.

5.3 Gain Changing

A simple script has been made to assist with the design of typical SPI communication required for the experiment. The example is on changing the gains. To run example script use following command on the box:

```
acq1001_695> /usr/local/CARE/set-device-gain.py
```

Follow instructions to change gain of desired sensor.

6 I2C Devices

The OCTO-BEE-CELF system includes two I^{2}C-controlled 12-bit Analog-to-Digital Converters (ADCs), both using the ADS7128 device. Each ADC operates with a 5 V reference voltage.

One ADC is used to read the Analog Output ASIC Temperature (AMUX) signals from the sensors, while the other monitors the internal regulated analog supply voltage (VCCA), which is 4.5 V.

Each temperature sensor corresponds to one of the eight input channels (0-7) on ADC 1. To read the raw ADC value from a specific channel, use the following commands:

```
cat /sys/bus/iio/devices/iio:device1/in_voltage0_raw
...
cat /sys/bus/iio/devices/iio:device1/in_voltage7_raw
```

Channels 0 through 7 represent the eight temperature sensor inputs connected to ADC 1.

Similarly, to read the voltage from each VCCA channel (0-7) on ADC 2, run:

```
cat /sys/bus/iio/devices/iio:device2/in_voltage0_raw
...
cat /sys/bus/iio/devices/iio:device2/in_voltage7_raw
```

Each command returns a 16-bit value, but only the upper 12 bits contain valid ADC data. To extract the 12-bit ADC code, shift the value right by 4 bits. For example:

```
# Raw binary data:
0b10111011 11010000

# Shift right by 4 bits:
0b00001011 10111011
```

6.1 Temperature Reading Example

Grab temperature data for sensor 1:

```
acq1001_695> cat /sys/bus/iio/devices/iio:device1/in_voltage0_raw
25344
```

Note: You can also obtain this value via the webpage or from a hexdump as explained later.

To shift down you can either convert to binary form and manually shift or in decimal format to shift right you do:

$$\begin{aligned} \text{shifted\_code} &= \frac{\text{orig\_bit\_code}}{2^{\text{bits\_shifted}}} \\ 12\text{bit\_code} &= \frac{16\text{bit\_code}}{2^4} \\ 12\text{bit\_code} &= \frac{25344}{16} \\ 12\text{bit\_code} &= 1584 \end{aligned} \tag{1}$$

Now to convert to volts:

$$\begin{aligned} \text{Voltage} &= \frac{\text{ADC}_{\text{CODE}} \times V_{\text{REF}}}{2^{\text{ADC}_{\text{RES}}}} \\ \text{Voltage} &= \frac{1584 \times 5}{2^{12}} \\ \text{Voltage} &= \frac{1584 \times 5}{2^{12}} \\ \text{Voltage} &= 1.93V \end{aligned} \tag{2}$$

To convert the ADC readings into temperature values, refer to Page 10, Table 11 of the *3DHALL Magnetic Sensor – SENM3Dx v2* datasheet. This table provides the relationship between the sensor's analog output voltage and the corresponding temperature. See table below:

Temperature sensor, analog and digital out	
Output voltage TA @0°C [V]	1.65
Sensitivity of TA, Range 15-40°C, [mV/°C]	10.30
Digital Temperature Reading Sensitivity, range 15-40°C [LSB/°C]	168
Digital Temperature Reading @0°C [LSB]	19913
Temperature sensor accuracy [°C]	< 0.1

Table 11: Temperature Sensor Specification

Figure 16: Temperature Sensor Data from SENIS Datasheet

Given:

- Measured voltage: $V_{\text{measured}} = 1.93$ V
- Sensor voltage at $0°$C: $V_{0°\text{C}} = 1.65$ V
- Sensor sensitivity: $S = 10.30$ mV/°C $= 0.01030$ V/°C
- Sensor range: $15 - 40$ °C

Lets first calculate the change in voltage from measured to 0°C

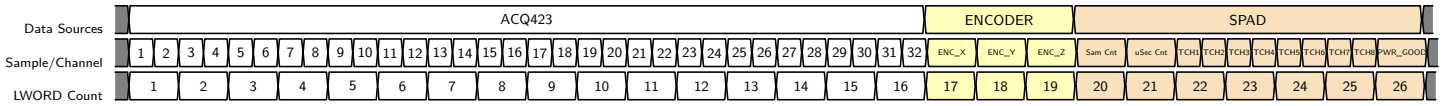
$$\begin{aligned} \Delta V &= V_{\text{measured}} - V_{0°\text{C}} \\ \Delta V &= 1.93 - 1.65 = 0.28 \text{ V} \end{aligned} \tag{3}$$

Now convert to temperature:

$$\begin{aligned} T &= \frac{\Delta V}{S} \\ T &= \frac{0.28}{0.01030} \approx 27.2 \text{ °C} \end{aligned} \tag{4}$$

7 Sample Data Format

The data packet when gathering data from the box will have the following format:



Signal	Description
Sam Cnt	Sample Counter from beginning of data capture.
uSec Cnt	Configurable 1 MHz (1 μs) counter.
ENC_X/Y/Z	Encoder A, B, Z data for 3 Axis.[1]
TCH1..8	16-bit Analog Output ASIC Temperature Reading.
PWR_GOOD	Power Good Monitoring for connected SENIS sensors.

Table 2: Signal Descriptions

```
dt100@danna:/home/projects/ACQ400/octave/ch_data$ timeout 10 nc acq1001_695 4210 | pv > octo-bee_695
125MiB 0:00:09 [17.8MiB/s] [                    <=>
dt100@danna:/home/projects/ACQ400/octave/ch_data$ hexdump -e '26/4 "%08x " "\n"' octo-bee_695 | cut -d ' ' -f1-4,11- | head
# CH1/2                 CH5/6    CH21/22                                 CH31/32  ENC X    ENC Y    ENC Z    Sam Cnt  uSec     T1/2     T3/4     T5/6     T7/8     PWR_GOOD
3846381b 385c3810 38b9389e 38c338c6 3bac3aa0 3ae43ac8 3bf23912 398a3a0b 41242c07 39735625 00000000 00000000 00000000 00000001 000000ff 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
38103826 385d3821 388f383a 38c238ab 3bb83ac0 3ae33a94 3c0e3943 398c3a2a 41352c2f 397955ea 00000000 00000000 00000000 00000002 000001f9 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
383b37ec 385e3801 387b3861 38c238bd 3bd03a89 3ae33ac2 3be63951 39893a1e 41172b84 397555fa 00000000 00000000 00000000 00000003 000002f3 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
383137b2 385e3828 388b3844 38c238bf 3bb43acd 3ae43a9d 3c09392d 398a3a27 40f82c0d 397155ff 00000000 00000000 00000000 00000004 000003ed 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
3835384a 385e37ff 38be3819 38c2386f 3bc33b0e 3ae43ac2 3c053951 398a3a2e 41442bc3 397455eb 00000000 00000000 00000000 00000005 000004e7 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
37e43812 385c37f9 388d384f 38c138be 3b953ab5 3ae53aa6 3bed392e 39893a23 411c2bd4 39795637 00000000 00000000 00000000 00000006 000005e1 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
3856381e 385e37ec 38873829 38c3388d 3b9a3ac6 3ae43a88 3bf93901 398a3a28 41112c04 3977561e 00000000 00000000 00000000 00000007 000006db 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
382237f2 385c3804 38e3387e 38c33899 3b853a7f 3ae33aba 3c173965 39893a36 40e32ba6 397555eb 00000000 00000000 00000000 00000008 000007d5 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
383237fd 385d381f 38ae3890 38c238d8 3b973ae1 3ae43a9e 3bd93920 398a3a1d 40f62bb3 397555da 00000000 00000000 00000000 00000009 000008cf 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
38663807 385c381d 3885384d 38c238dd 3b7d3aa8 3ae33aa1 3bcd3963 398b3a2f 410a2c0d 397955d9 00000000 00000000 00000000 0000000a 000009c9 5c605b10 626060e0 65b05f00 60305fa0 ffffffff
```

See code snippet above.

The first command uses ‘nc’ (netcat) to connect to the remote data acquisition unit ‘acq1001_695’ on port ‘4210’, streaming raw binary data from it. The output is piped through ‘pv’, which shows transfer progress, and redirected into a file named ‘octo-bee_695’.

The ‘timeout 10’ specifies to only stream 10 seconds worth of data.

The second command inspects this binary file using ‘hexdump’, which prints the data in hexadecimal format. The format string ‘-e ‘26/4 “%08x ” “\n” ’ ’ groups and displays the 26 4-byte longwords per line in 8-digit hexadecimal.

This is then piped to ‘cut’ to keep only specific fields (columns 1–4 and 11 onward), reducing the line width for readability. To dump all channels remove “ cut -d ‘ ’ -f1-4,11- | ” and pipe straight onto head. Finally piping to ‘head’ makes it only show the first ten lines.

These two commands captures a stream of raw digitized data from the system, saves it locally, and displays a trimmed, readable hex snapshot of the first few data records.

Looking at the last column “PWR_GOOD”, which currently shows FFFFFFFF, indicates that all eight sensors are connected and operating correctly. Monitoring this value makes it easy to identify any faulty or disconnected sensors during debugging. This value only updates on a reboot.

A Phoebus Full Install Guide

A.1 Git

A.1.1 Install Git (if required)

Install Git at https://git-scm.com/downloads/win

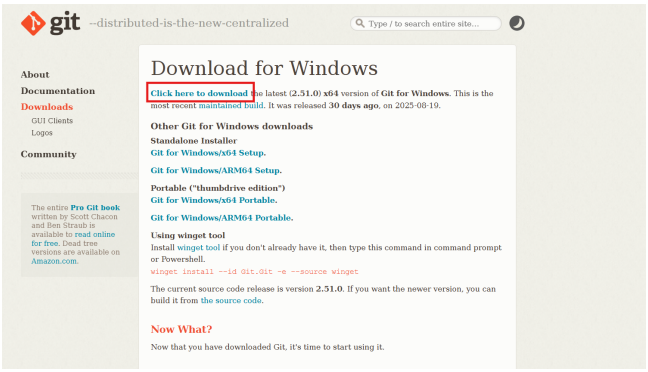
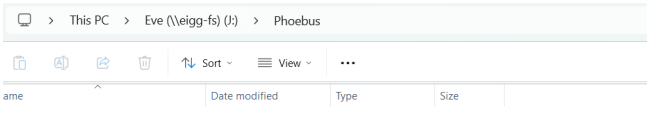


Figure 17: Where to install git

Click installer to install and use default settings.

A.1.2 Open Git Bash

Hold shift and right click in desired folder where you want our GUI to be located and click “Open Git Bash Here”



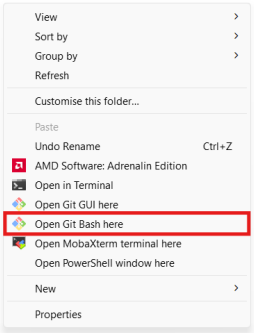


Figure 18: Git Bash

A.1.3 Git Clone Package

In the git command terminal type:

```
git clone https://github.com/D-TACQ/ACQ400CSSP
```

see desired output:

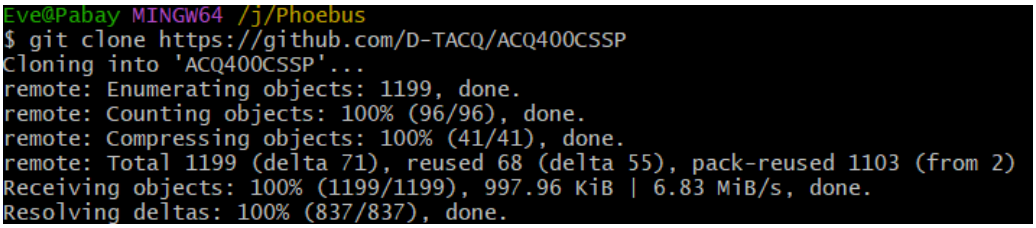


Figure 19: Install Package

Now run

```
cd ACQ400CSSP
```

A.2 Install Java (If required)

Go to Adoptium Java 17 downloads and install Java:

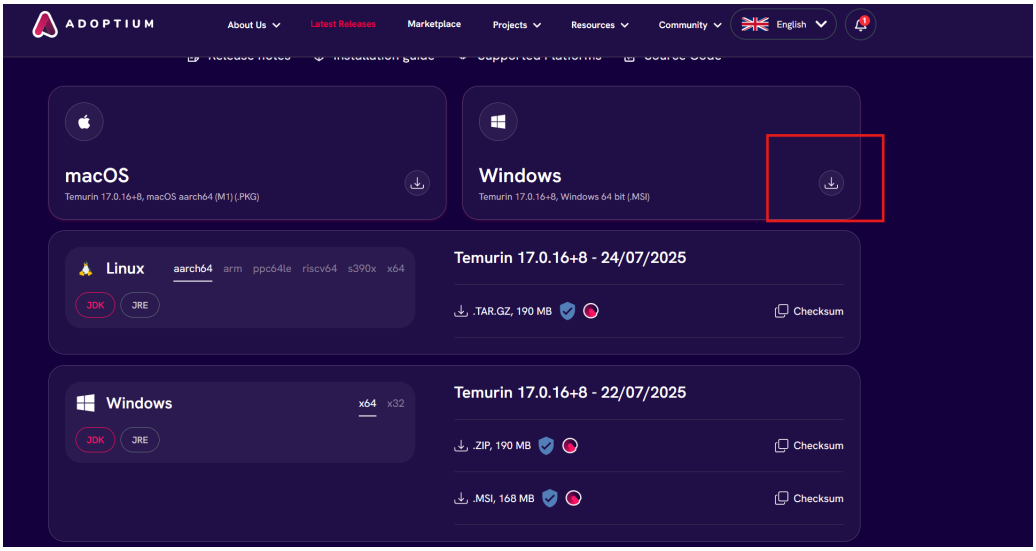


Figure 20: Install Java

Use default settings.

A.3 Installing Phoebus

In your git bash terminal run:

```
curl -LO https://github.com/ControlSystemStudio/phoebus/releases/download/v5.0.2/phoebus-5.0.2-win.zip
unzip phoebus-5.0.2-win.zip
```

A.4 Installing Phoebus (Manually)

A.4.1 Download and Extract Phoebus

Download and extract the zip file: Phoebus Download

A.4.2 Configure Directory

Once extracted copy/move the folder “product-5.0.2” into the folder ACQ400CSSP

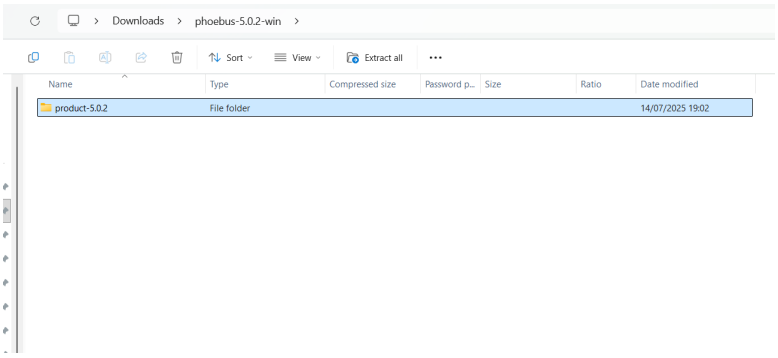


Figure 21: Folder to move

Your “ACQ400CSSP” directory should now look like this:

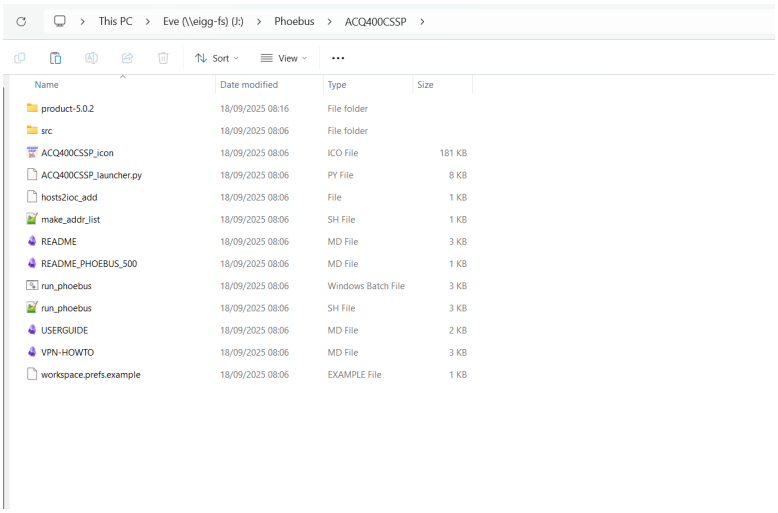


Figure 22: Desired Directory Structure

A.4.3 Make Conf File (Only if Needed - Try without first)

If experiencing errors when running due to the Java not being mapped properly come back to this section and edit the CSSP.conf file as below:

```
JAVA_BIN=C:\Program Files\Eclipse Adoptium\jre-17.0.12.7-hotspot\bin\java.exe
PHOEBUS_JAR=C:\Users\Eve\Phoebus\ACQ400CSSP\product-5.0.2\product-5.0.2.jar
```

Note: your file path for “java.exe” may be different, try to find this file yourself to confirm the file path and edit accordingly.

Your filepath for where you “product-5.0.2.jar” will definitely be different. Please change this file path to where you have saved this folder.

A.4.4 Install Python (if required)

Download Python here.

Figure 23: Python Install Page

A.4.5 Create Shortcut

Shift right click file ACQ400CSSP_launcher.py and select “Create Shortcut”

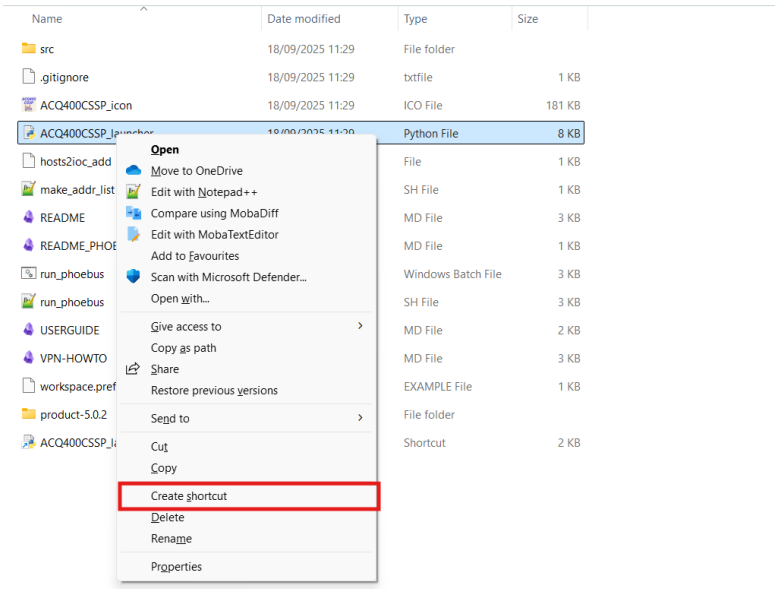


Figure 24: Create Shortcut of Launcher

Right click on this shortcut and click on “Properties”.

In the “Target” field go to the very left and type Python3 before the existing path:

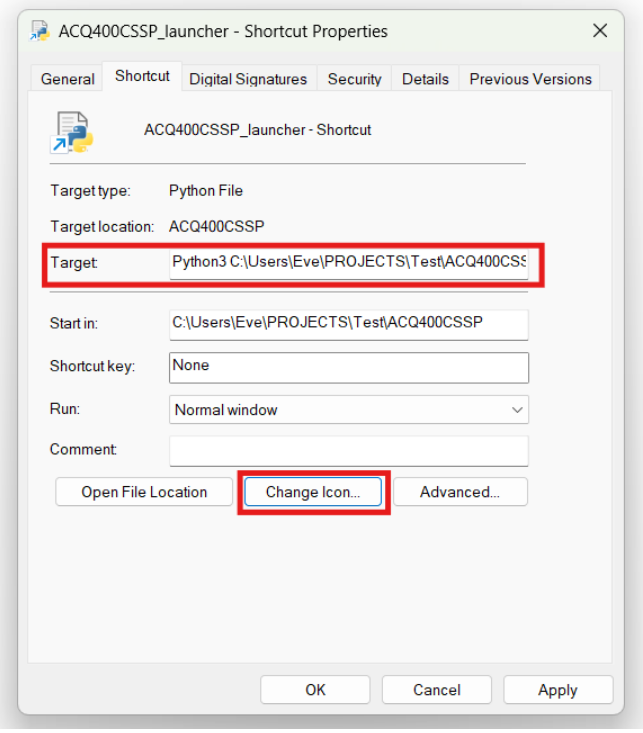


Figure 25: Launcher Properties

Click on “Change Icon...”, Press “Browse...” and Open “ACQ400CSSP_icon” found in the ACQ400CSSP folder.

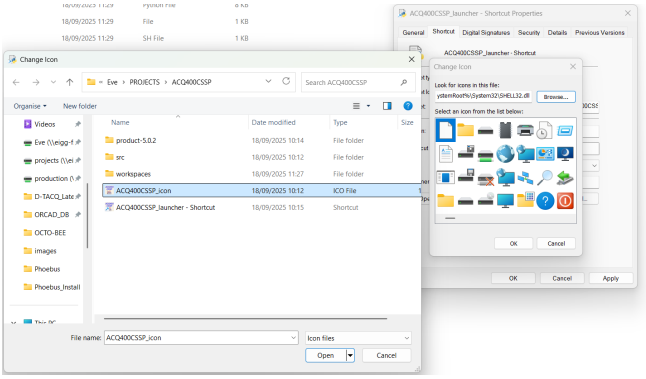


Figure 26: Change Icon

Back on properties press “Apply” and “Ok” to close.

Now you have made this shortcut shift right-click and select “Pin to taskbar” if you wish to have easy access.

All the necessary installation is now done.

A.5 Run / Open Phoebus

You can now press this shortcut pinned to task bar and enter your boards serial number and it will open up Phoebus for this device.

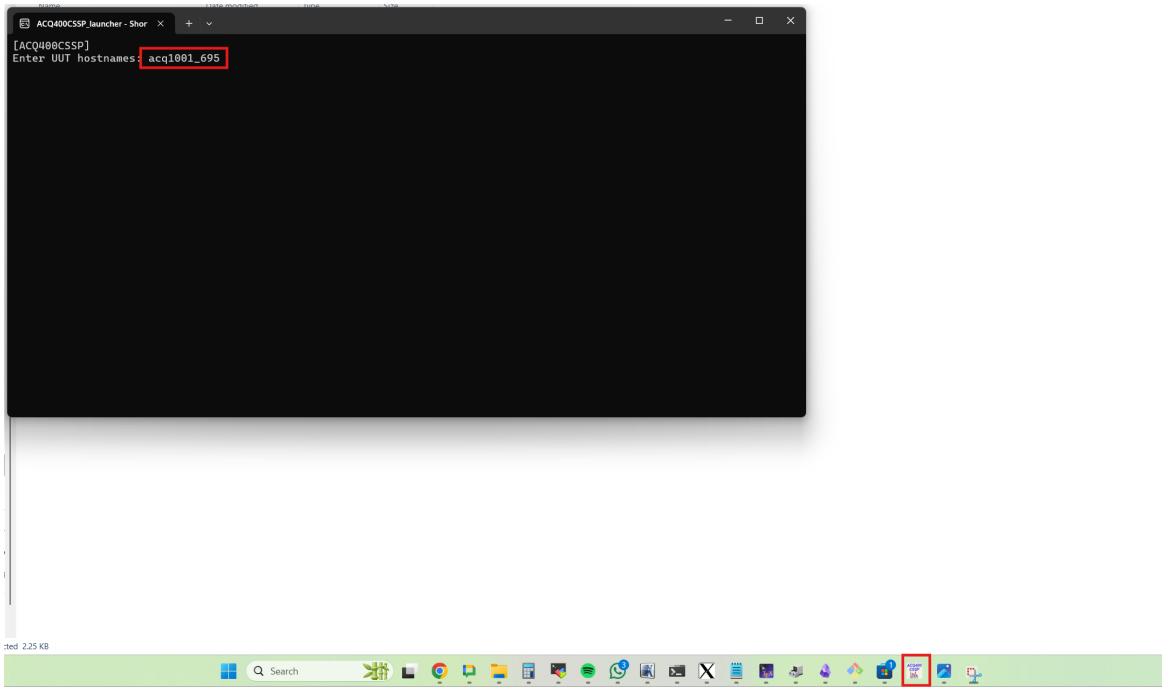


Figure 27: Open Phoebus

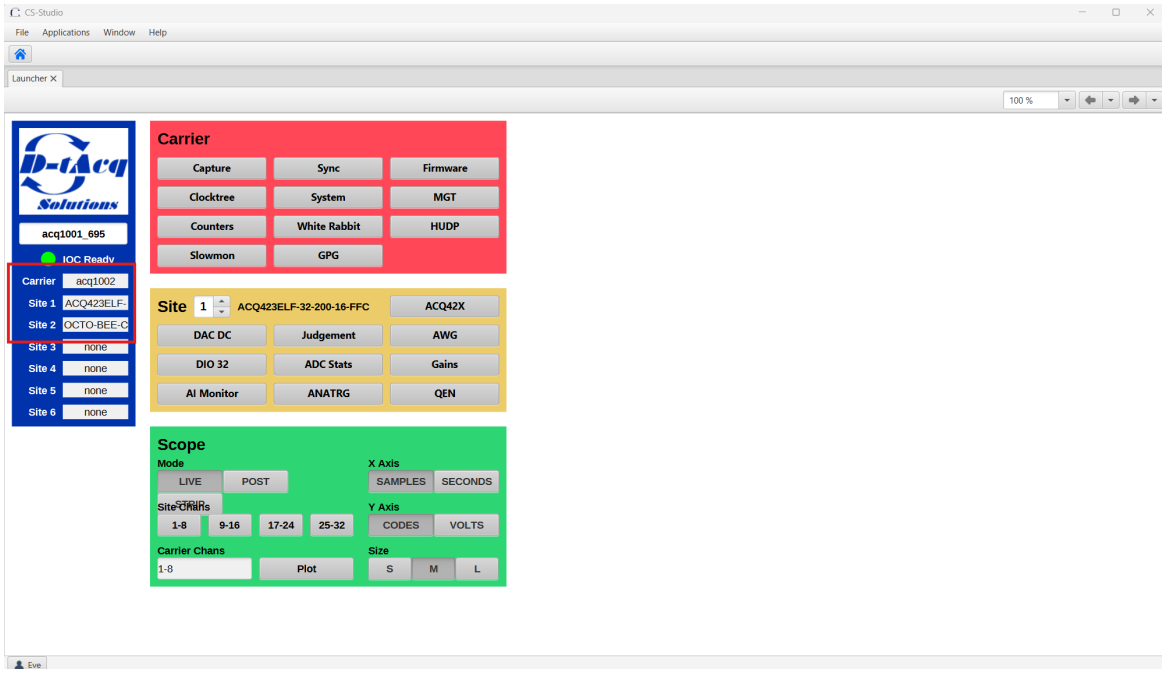


Figure 28: Run Phoebus

Confirm everything looks right on your sites.

B OCTO-BEE Gold System How-to

B.1 Firmware Update

Download latest release found here

Follow instructions on ACQ400RELEASE found here on how to deploy the latest firmware onto your outdated box.

B.2 Set Device Tree

The OCTO-BEE needs a special device tree to set the quadrature encoder sites up.

Connect to the system with serial console. (PuTTY) Hit spacebar within the first three seconds on boot to enter the u-boot prompt.

```
Xilinx First Stage Boot Loader
Release 2015.2  Sep 14 2015-10:44:27
Silicon Version 3.1
Boot mode is QSPI
................................................................................
SUCCESSFUL_HANDOFF
FSBL Status = 0x1


U-Boot 2013.04-00003-g8e84774 (Nov 22 2013 - 10:21:51)

I2C:   ready
Memory: ECC disabled
DRAM:  1 GiB
WARNING: Caches not enabled
MMC:   zynq_sdhci: 0
SF: Detected N25Q128A with page size 64 KiB, total 16 MiB
In:    serial
Out:   serial
Err:   serial
Net:   Gem.e000b000, Gem.e000c000
Press the spacebar to stop ACQ2006 autoboot:  0

acq2006-uboot>

acq2006-uboot> printenv devicetree_image
devicetree_image=dtb.d/acq1002.dtb
acq2006-uboot> env set devicetree_image "dtb.d/acq1002octo.dtb"
acq2006-uboot> printenv devicetree_image
devicetree_image=dtb.d/acq1002octo.dtb
acq2006-uboot> saveenv
acq2006-uboot> boot
```

B.3 Packages/rc.user/transient.init

B.3.1 PMOD Package

Be sure to move package ”04-custom_pmod-2601301528.tgz” from /mnt/packages.opt to /mnt/packages

B.3.2 Custom OCTOBEE Package

Available as a .tgz from Github here

```
acq1001_695> ls -l $PWD/99-custom_octobee-2601301630.tgz
-rwxr-xr-x    1 root     root         93440 Jan 30 16:31 /mnt/packages/99-custom_octobee-2601301630.tgz
```

B.3.3 rc.user

```
# DEFAULT init for acq423, 200000 Hz, auto soft trigger
#
# created by deploy_sysconfig for uut:acq1001_695 mezz:acq423
# by dt100@eigg-fs on Thu 11 Sep 15:11:15 BST 2025
# git dfd3753558f321e924fae47dea28d1803192bfa4
# incant ./deploy_sysconfig.sh acq1001_695 acq423 1

/usr/local/CARE/acq1001\+acq42x.init
# Local clock and soft trigger
set.site 0 sync_role master 200000
# Alternate front panel clock and front panel trigger
#set.site 0 sync_role fpmaster 200000 1000000

# For other combinations try 'set.site 0 sync_role help'

judgement() {
# short trace length, rapid update 50Hz possible
set.site 1 RTM_TRANSLEN ${1:-128}
set.site 1 RGM RTM
set.site 1 RGM:DX ${2:-d0}
set.site 1 RGM:SENSE rising
}
# see /mnt/local/sysconfig/epics.sh
source /mnt/local/sysconfig/acq400.sh
[ ! -z "$ACQ400_JUDGEMENT" ] && judgement $ACQ400_JUDGEMENT

# Set trigger source for SPAD uSec counter; soft_trigger same as ADC default trigger
set.site 0 spad1_us 1,1,1

# Enable X, Y, Z Quadrature Encoder counts
for sites in 2 5 6; do
  set.site $sites phaseA_en 1
  set.site $sites phaseB_en 1
done

# Power good monitoring for connected SENIS sensors
# Checks once upon boot
/mnt/local/pgood_spad

# Enable I2C temperature ADC poll and include in SPAD2-6
/mnt/local/temps_spad &

# enable port 4210 stream
echo STREAM_OPTS= >> /etc/sysconfig/acq400_streamd.conf
export PYTHONPATH=$PYTHONPATH:/usr/local/senm3dx

# Set sensor gains
for sensor in 0 1 2 3 4 5 6 7; do
    /usr/local/CARE/set-device-gain.py $sensor 3000
done

# set sites in aggregator set
# Site 1 - ACQ423, Site 2,5,6 - Quadrature Encoder, 7 LWord SPAD
run0 1,2,5,6 1,7,0
```

B.3.4 transient.init

```
# NSAMPLES sets EPICS transient WF length
# next line: expect condition to stay this boot
COOKED=0 NSAMPLES=100000 NCHAN=32 TYPE=SHORT

# set a default transient. expect this to change at run time
transient PRE=2000 POST=2000 OSAM=1 DEMUX=1 SOFT_TRIGGER=1

# configure soft trigger on site 1 - typical default
set.site 1 trg=1,1,1

# add the number of active channels in site 2
echo 1 > /etc/acq400/2/active_chan
echo 1 > /etc/acq400/2/data32
echo 1 > /etc/acq400/5/active_chan
echo 1 > /etc/acq400/5/data32
echo 1 > /etc/acq400/6/active_chan
echo 1 > /etc/acq400/6/data32

# set sites in aggregator set
# Site 1 - ACQ423, Site 2,5,6 - Quadrature Encoder, 7 LWord SPAD
run0 1,2,5,6 1,7,0

# configure transient channel data server at ports 53000+CH
make-ch-server
```

C Fake site configuration

Required files

```
acq1001_695> ls -l $PWD/*.conf
-rwxr-xr-x    1 root     root            19 Jan 23 13:15 /mnt/local/custom_pmod.conf
-rwxr-xr-x    1 root     root            30 Jan 23 13:15 /mnt/local/fmc-scan.conf
acq1001_695> ls -l $PWD/*.fru
-rwxr-xr-x    1 root     root           240 Feb  3 15:28 /mnt/local/CUSTOM_PMOD_5_AC4060999.fru
-rwxr-xr-x    1 root     root           240 Feb  3 15:31 /mnt/local/CUSTOM_PMOD_6_AC4060999.fru
acq1001_695> cat custom_pmod.conf
CUSTOM_PMODS="5 6"
acq1001_695> cat fmc-scan.conf
FMC_SCAN_SITES="1 2 3 4 5 6"
```

D FPGA configuration

Custom .bd file to enable extra site register decodes in ACQ1001

`ACQ1001_6SITE_FSBL`